Contents

The Sonnet Compiler - written in C II
Boilerplate II
Imports II
Declarations II
String II
The Ast IV
Lexer X
Pretty Printing XVI
Main XIX

The Sonnet Compiler - written in C

Boilerplate

Imports

```
#include <ctype.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Declarations

```
typedef struct String String;

typedef enum TokenType TokenType;
typedef struct Token Token;
typedef struct TokenStack TokenStack;
```

String

```
struct String
    {
    char* data;
    size_t length;
    size_t capacity;
    };

void string_init(String* s, const char* initial)
    {
    size_t len = strlen(initial);
    s->length = len;
    s->capacity = len;
    s->data = (char*)malloc(s->capacity);
    strcpy(s->data, initial);
    }

String string_empty()
    {
    String s;
    s.data = (char*)malloc(1);
    s.data[0] = '\0';
    s.length = 0;
    s.capacity = 1;
    return s;
    }

const char* string_get_data(const String* s) { return s->data; }

void string_append(String* s, const char* str)
    {
```

```
    size_t new_len = s->length + strlen(str);

    if (new_len >= s->capacity)
        {
        s->capacity = new_len + 16;
        s->data = (char*)realloc(s->data, s->capacity);
        }

    strcat(s->data, str);
    s->length = new_len;
    }

void string_push_char(String* s, char c)
    {
    if (s->length + 1 >= s->capacity)
        {
        s->capacity = s->capacity * 2 + 16;
        s->data = (char*)realloc(s->data, s->capacity);
        }

    s->data[s->length] = c;
    s->length++;
    s->data[s->length] = '\0';
    }

void string_free(String* s)
    {
    free(s->data);
    s->data = NULL;
    s->length = 0;
    s->capacity = 0;
    }

String read_file(const char* filename)
    {
    String s = string_empty();
    FILE* f = fopen(filename, "r");

    if (f)
        {
        char buffer[1024];
        while (fgets(buffer, sizeof(buffer), f))
            {
            string_append(&s, buffer);
            }
        fclose(f);
        }
    return s;
    }
```

The Ast

```
typedef struct Ast Ast;
typedef enum TypeType TypeType;
typedef struct Type Type;
typedef enum NodeType NodeType;

enum TypeType
    {
    T_I8,
    T_I16,
    T_I32,
    T_I64,
    T_U8,
    T_U16,
    T_U32,
    T_U64,
    T_F16,
    T_F32,
    T_F64,

    T_Struct,
    T_Union,
    T_Enum,
    T_Pointer
    };

struct Type
    {
    TypeType tag;
        union {
        struct
            {
            struct
                {
                String name;
                Type* type;
                }* fields;
            size_t field_count;
            } struct_type;
        struct
            {
            struct
                {
                String name;
                Type* type;
                }* fields;
            size_t field_count;
            } union_type;
        struct
            {
            struct
                {
                String name;
                }* fields;
            size_t field_count;
```

```
            } enum_type;
        };
    };

typedef enum
{
    ADD,
    SUB,
    DIV,
    MUL,
    LessThan,
    LessThanEqual,
    MoreThan,
    MoreThanEqual,
    Equal,
    NotEqual
} BinOpType;

enum NodeType
    {
    Var,
    BinOp,
    Fun,
    App,
    Struct,
    Union,
    Enum,
    Compound,
    VarDecl,
    VarSet,
    Loop,
    Ref,
    Deref,
    SetPtr,
    AccessPtr,
    I32Lit,
    StringLit
    };

struct Ast
    {
    NodeType type;
        union {
        struct
            {
            String name;
            } var;
        struct
            {
            BinOpType op;
            Ast *left, *right;
            } bin_op;
        struct
            {
            String name;
            struct
```

```
            {
            String name;
            Type* type;
            }* params;
        size_t param_count;
        Type* return_type;
        Ast* body;
        } fun;
    struct
        {
        Ast* callee;
        Ast** args;
        size_t arg_count;
        } app;
    struct
        {
        struct
            {
            String name;
            Ast* val;
            }* fields;
        size_t field_count;
        } struct_val;
    struct
        {
        struct
            {
            String name;
            Ast* val;
            }* fields;
        size_t field_count;
        } union_val;
    struct
        {
        struct
            {
            String name;
            }* fields;
        size_t field_count;
        } enum_val;
    struct
        {
        Ast** exprs;
        size_t expr_count;
        } compound;
    struct
        {
        String name;
        Type* annot;
        Ast* init_expr;
        } var_decl;
    struct
        {
        String name;
        Ast* expr;
        } var_set;
```

```
        struct
            {
            Ast* expr;
            } loop;
        struct
            {
            Ast* expr;
            } ref;
        struct
            {
            Ast* expr;
            } deref;
        struct
            {
            Ast* target;
            Ast* value;
            } set_ptr;
        struct
            {
            Ast* target;
            String member;
            } access_ptr;
        struct
            {
            int32_t value;
            } i32_lit;
        struct
            {
            String value;
            } string_lit;
        } data;
    };

Ast* new_node(Ast ast)
    {
    Ast* ptr = malloc(sizeof(Ast));
    if (ptr)
        *ptr = ast;
    return ptr;
    }

void type_free(Type* t)
    {
    if (!t)
        return;

    if (t->tag == T_Struct || t->tag == T_Union)
        {
        for (size_t i = 0; i < t->struct_type.field_count; i++)
            {
            string_free(&t->struct_type.fields[i].name);
            type_free(t->struct_type.fields[i].type);
            }
        free(t->struct_type.fields);
        }
    else if (t->tag == T_Enum)
```

```
        {
        for (size_t i = 0; i < t->enum_type.field_count; i++)
            {
            string_free(&t->enum_type.fields[i].name);
            }
        free(t->enum_type.fields);
        }
    free(t);
    }

void ast_free(Ast* node)
    {
    if (!node)
        return;

    switch (node->type)
        {
    case Var:
        string_free(&node->data.var.name);
        break;

    case BinOp:
        ast_free(node->data.bin_op.left);
        ast_free(node->data.bin_op.right);
        break;

    case Fun:
        string_free(&node->data.fun.name);
        for (size_t i = 0; i < node->data.fun.param_count; i++)
            {
            string_free(&node->data.fun.params[i].name);
            type_free(node->data.fun.params[i].type);
            }
        free(node->data.fun.params);
        type_free(node->data.fun.return_type);
        ast_free(node->data.fun.body);
        break;

    case App:
        ast_free(node->data.app.callee);
        for (size_t i = 0; i < node->data.app.arg_count; i++)
            {
            ast_free(node->data.app.args[i]);
            }
        free(node->data.app.args);
        break;

    case Struct:
        for (size_t i = 0; i < node->data.struct_val.field_count; i++)
            {
            string_free(&node->data.struct_val.fields[i].name);
            ast_free(node->data.struct_val.fields[i].val);
            }
        free(node->data.struct_val.fields);
        break;
```

```
    case Union:
        for (size_t i = 0; i < node->data.union_val.field_count; i++)
            {
            string_free(&node->data.union_val.fields[i].name);
            ast_free(node->data.union_val.fields[i].val);
            }
        free(node->data.union_val.fields);
        break;

    case Enum:
        for (size_t i = 0; i < node->data.enum_val.field_count; i++)
            {
            string_free(&node->data.enum_val.fields[i].name);
            }
        free(node->data.enum_val.fields);
        break;

    case Compound:
        for (size_t i = 0; i < node->data.compound.expr_count; i++)
            {
            ast_free(node->data.compound.exprs[i]);
            }
        free(node->data.compound.exprs);
        break;

    case VarDecl:
        string_free(&node->data.var_decl.name);
        type_free(node->data.var_decl.annot);
        ast_free(node->data.var_decl.init_expr);
        break;

    case VarSet:
        string_free(&node->data.var_set.name);
        ast_free(node->data.var_set.expr);
        break;

    case Loop:
        ast_free(node->data.loop.expr);
        break;

    case Ref:
        ast_free(node->data.ref.expr);
        break;

    case Deref:
        ast_free(node->data.deref.expr);
        break;

    case SetPtr:
        ast_free(node->data.set_ptr.target);
        ast_free(node->data.set_ptr.value);
        break;

    case AccessPtr:
        ast_free(node->data.access_ptr.target);
        string_free(&node->data.access_ptr.member);
```

```
        break;

    case I32Lit:
        break;

    case StringLit:
        string_free(&node->data.string_lit.value);
        break;
        }

    free(node);
    }
```

Lexer

```
enum TokenType
    {
    TOK_EOF,
    TOK_IDENT,
    TOK_INT,
    TOK_STRING,
    TOK_ASSIGN,
    TOK_LEFT_ARROW,
    TOK_RIGHT_ARROW,
    TOK_DOT,
    TOK_STAR,
    TOK_EXCLAMATION,
    TOK_LPAREN,
    TOK_RPAREN,
    TOK_LBRACE,
    TOK_RBRACE,
    TOK_COMMA,
    TOK_KW_FUN,
    TOK_KW_LET,
    TOK_KW_IF,
    TOK_KW_THEN,
    TOK_KW_ELSE,
    TOK_KW_DEFTYPE,
    TOK_KW_STRUCT,
    TOK_KW_ENUM,
    TOK_KW_UNION,
    TOK_PLUS,
    TOK_MINUS,
    TOK_LESS,
    TOK_LESS_EQ,
    TOK_MORE,
    TOK_MORE_EQ,
    TOK_EQ,
    TOK_NOT_EQ
    };

struct Token
```

```
    {
    TokenType type;
        union {
        String ident_or_lit;
        int32_t lit;
        } dat;
    };

Token tok_empty(TokenType t)
    {
    Token tok;
    tok.type = t;
    return tok;
    }

struct TokenStack
    {
    Token* tokens;
    size_t top;
    size_t capacity;
    };

void stack_free(TokenStack* s)
    {
    if (!s || !s->tokens)
        return;

    for (size_t i = 0; i < s->top; i++)
        {
        Token* t = &s->tokens[i];

        if (t->type == TOK_IDENT || t->type == TOK_STRING)
            {
            string_free(&t->dat.ident_or_lit);
            }
        }

    free(s->tokens);

    s->tokens = NULL;
    s->top = 0;
    s->capacity = 0;
    }

TokenStack stack_empty()
    {
    TokenStack s;
    s.tokens = NULL;
    s.top = 0;
    s.capacity = 0;
    return s;
    }

void stack_push(TokenStack* s, Token t)
    {
    if (s->top >= s->capacity)
```

```
        {
        s->capacity = s->capacity == 0 ? 16 : s->capacity * 2;
        s->tokens = realloc(s->tokens, sizeof(Token) * s->capacity);
        }
    s->tokens[s->top++] = t;
    }

Token stack_pop(TokenStack* s)
    {
    if (s->top == 0)
        return (Token){TOK_EOF};
    return s->tokens[--s->top];
    }

Token stack_peek(TokenStack* s)
    {
    if (s->top == 0)
        return tok_empty(TOK_EOF);
    return s->tokens[s->top - 1];
    }

void stack_reverse(TokenStack* s)
    {
    if (s->top == 0)
        return;

    TokenStack s2 = stack_empty();

    while (s->top != 0)
        {
        stack_push(&s2, stack_pop(s));
        }

    Token* temp_tokens = s->tokens;
    s->tokens = s2.tokens;
    s2.tokens = temp_tokens;

    size_t temp_top = s->top;
    s->top = s2.top;
    s2.top = temp_top;

    size_t temp_cap = s->capacity;
    s->capacity = s2.capacity;
    s2.capacity = temp_cap;

    free(s2.tokens);
    }

int is_ident_char(char c) { return isalnum(c) || c == '_'; }

TokenType get_keyword_type(const char* str)
    {
    if (strcmp(str, "fun") == 0)
        return TOK_KW_FUN;
    if (strcmp(str, "let") == 0)
        return TOK_KW_LET;
```

```
    if (strcmp(str, "if") == 0)
        return TOK_KW_IF;
    if (strcmp(str, "then") == 0)
        return TOK_KW_THEN;
    if (strcmp(str, "else") == 0)
        return TOK_KW_ELSE;
    if (strcmp(str, "deftype") == 0)
        return TOK_KW_DEFTYPE;
    if (strcmp(str, "struct") == 0)
        return TOK_KW_STRUCT;
    if (strcmp(str, "enum") == 0)
        return TOK_KW_ENUM;
    if (strcmp(str, "union") == 0)
        return TOK_KW_UNION;
    return TOK_IDENT;
    }

void lex(const String* input, TokenStack* stack)
    {
    size_t i = 0;
    const char* data = input->data;
    size_t len = input->length;

    while (i < len)
        {
        char c = data[i];

        if (isspace(c))
            {
            i++;
            continue;
            }

        if (isalpha(c) || c == '_')
            {
            String ident = string_empty();
            while (i < len && is_ident_char(data[i]))
                {
                string_push_char(&ident, data[i]);
                i++;
                }

            TokenType type = get_keyword_type(ident.data);
            Token t = tok_empty(type);

            if (type == TOK_IDENT)
                {
                t.dat.ident_or_lit = ident;
                }
            else
                {
                string_free(&ident);
                }

            stack_push(stack, t);
            continue;
```

```
        }

    if (isdigit(c))
        {
        int32_t val = 0;
        while (i < len && isdigit(data[i]))
            {
            val = val * 10 + (data[i] - '0');
            i++;
            }
        Token t = tok_empty(TOK_INT);
        t.dat.lit = val;
        stack_push(stack, t);
        continue;
        }

    if (c == '"')
        {
        i++;
        String str_val = string_empty();

        while (i < len && data[i] != '"')
            {
            string_push_char(&str_val, data[i]);
            i++;
            }

        if (i < len && data[i] == '"')
            {
            i++;
            }

        Token t = tok_empty(TOK_STRING);
        t.dat.ident_or_lit = str_val;
        stack_push(stack, t);
        continue;
        }

    Token t;
    switch (c)
        {
    case '=':
        if (i + 1 < len && data[i + 1] == '=')
            {
            t.type = TOK_EQ;
            i += 2;
            }
        else
            {
            t.type = TOK_ASSIGN;
            i++;
            }
        stack_push(stack, t);
        break;
    case '<':
        if (i + 1 < len && data[i + 1] == '=')
```

```
                {
                t.type = TOK_LESS_EQ;
                i += 2;
                }
            else if (i + 1 < len && data[i + 1] == '-')
                {
                t.type = TOK_LEFT_ARROW;
                i += 2;
                }
            else
                {
                t.type = TOK_LESS;
                i++;
                }
            stack_push(stack, t);
            break;
        case '>':
            if (i + 1 < len && data[i + 1] == '=')
                {
                t.type = TOK_MORE_EQ;
                i += 2;
                }
            else
                {
                t.type = TOK_MORE;
                i++;
                }
            stack_push(stack, t);
            break;
        case '-':
            if (i + 1 < len && data[i + 1] == '>')
                {
                t.type = TOK_RIGHT_ARROW;
                i += 2;
                }
            else
                {
                t.type = TOK_MINUS;
                i++;
                }
            stack_push(stack, t);
            break;
        case '!':
            if (i + 1 < len && data[i + 1] == '=')
                {
                t.type = TOK_NOT_EQ;
                i += 2;
                }
            else
                {
                t.type = TOK_EXCLAMATION;
                i++;
                }
            stack_push(stack, t);
            break;
        case '+':
```

```
            t.type = TOK_PLUS;
            i++;
            stack_push(stack, t);
            break;
        case '.':
            t.type = TOK_DOT;
            i++;
            stack_push(stack, t);
            break;
        case '*':
            t.type = TOK_STAR;
            i++;
            stack_push(stack, t);
            break;
        case '(':
            t.type = TOK_LPAREN;
            i++;
            stack_push(stack, t);
            break;
        case ')':
            t.type = TOK_RPAREN;
            i++;
            stack_push(stack, t);
            break;
        case '{':
            t.type = TOK_LBRACE;
            i++;
            stack_push(stack, t);
            break;
        case '}':
            t.type = TOK_RBRACE;
            i++;
            stack_push(stack, t);
            break;
        case ',':
            t.type = TOK_COMMA;
            i++;
            stack_push(stack, t);
            break;
        default:
            i++;
            break;
            }
        }

    Token eof_tok;
    eof_tok.type = TOK_EOF;
    stack_push(stack, eof_tok);
    }
```

Pretty Printing

We define prtty printing functions for every object to easily inspect for debugging purposes:

```c
void print_indent(int depth)
    {
    for (int i = 0; i < depth; i++)
        printf("\t");
    }
```

Pretty Printing for the lexer and its tokens.

```c
void print_token(Token t)
    {
    switch (t.type)
        {
    case TOK_EOF:
        printf("EOF");
        break;
    case TOK_IDENT:
        printf("IDENT(\"%s\")", t.dat.ident_or_lit.data);
        break;
    case TOK_INT:
        printf("INT(%d)", t.dat.lit);
        break;
    case TOK_STRING:
        printf("STRING(\"%s\")", t.dat.ident_or_lit.data);
        break;
    case TOK_ASSIGN:
        printf("=");
        break;
    case TOK_LEFT_ARROW:
        printf("←");
        break;
    case TOK_RIGHT_ARROW:
        printf("→");
        break;
    case TOK_DOT:
        printf(".");
        break;
    case TOK_STAR:
        printf("*");
        break;
    case TOK_EXCLAMATION:
        printf("!");
        break;
    case TOK_LPAREN:
        printf("(");
        break;
    case TOK_RPAREN:
        printf(")");
        break;
    case TOK_LBRACE:
        printf("{");
        break;
    case TOK_RBRACE:
        printf("}");
```

```
        break;
    case TOK_COMMA:
        printf(",");
        break;
    case TOK_KW_FUN:
        printf("KW_FUN");
        break;
    case TOK_KW_LET:
        printf("KW_LET");
        break;
    case TOK_KW_IF:
        printf("KW_IF");
        break;
    case TOK_KW_THEN:
        printf("KW_THEN");
        break;
    case TOK_KW_ELSE:
        printf("KW_ELSE");
        break;
    case TOK_KW_DEFTYPE:
        printf("KW_DEFTYPE");
        break;
    case TOK_KW_STRUCT:
        printf("KW_STRUCT");
        break;
    case TOK_KW_ENUM:
        printf("KW_ENUM");
        break;
    case TOK_KW_UNION:
        printf("KW_UNION");
        break;
    case TOK_PLUS:
        printf("+");
        break;
    case TOK_MINUS:
        printf("-");
        break;
    case TOK_LESS:
        printf("<");
        break;
    case TOK_LESS_EQ:
        printf("≤");
        break;
    case TOK_MORE:
        printf(">");
        break;
    case TOK_MORE_EQ:
        printf("≥");
        break;
    case TOK_EQ:
        printf("==");
        break;
    case TOK_NOT_EQ:
        printf("≠");
        break;
        }
```

```
    printf("\n");
    }
```

Main

```
int main(int argc, char** argv)
    {
    if (argc < 2)
        {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
        }

    String source_code = read_file(argv[1]);

    if (source_code.length == 0)
        {
        printf("Error: Could not read file '%s' or file is empty.\n",
argv[1]);
        string_free(&source_code);
        return 1;
        }

    TokenStack stack = stack_empty();
    lex(&source_code, &stack);
    string_free(&source_code);

    stack_reverse(&stack);

    print_token(stack_pop(&stack));
    print_token(stack_pop(&stack));
    print_token(stack_pop(&stack));

    stack_free(&stack);

    return 0;
    }
```